- *Computational cost*—Running full LLM inference for each test can be time consuming and expensive.

To address some of these challenges, we can employ mocking, semantic similarity scoring, and constraint validation.

For writing test cases, we use the Jest framework due to its simplicity and ecosystem. The test cases focus on validating AI-generated text through two approaches: semantic similarity and constraint checking. The first measures how closely the generated response matches an expected output using cosine similarity, ensuring the meaning remains consistent. The second test verifies that the output respects specified constraints, such as word count limits and the inclusion of certain keywords.

Mocking LLM responses

Instead of calling the actual LLM for every test, we can mock the responses. The following listing shows an example of mocking the `generateText` function from the `ai` package.

Listing 5.2 Mock testing the prompt

```
import { vi, expect, test } from "vitest";
vi.mock("ai", () => ({
  generateText: vi.fn(),
}));

import { generateText } from "ai";
import { continueConversation } from "./actions";
test('summary generation contains key points', async () => {
  generateText.mockResolvedValue(
"A concise summary of the input text.");

  const result = await continueConversation([
    { role: "user", content: "Long input text..." },
  ]);

  expect(result).toContain("concise summary");
});
```

vitest mocks the entire ai module, replacing the generateText export with mock functions.

Imports the actual function we want to test from our application code

Creates a typed mock function for generateText, allowing TypeScript to provide type checking and autocompletion

Calls the actual function we're testing, which internally uses the mocked generateText

Running the example code inside the ch05 folder

To run the tests locally, first install the dependencies within the ch05 folder:

```
npm install
```

Then run all tests once:

```
npx vitest run
```